

Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 6a - Planning and Learning Under Partial Information

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Learning under Partial Observability

Partial Observability and Partial Knowledge

So far, we have assumed that the agent does not know the true state. But what exactly is the **state**?

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Partial Observability and Partial Knowledge

So far, we have assumed that the agent does not know the true state. But what exactly is the **state**?

- A state is a representation containing all information relevant for decision making.
- It may include:
 - physical properties (location, velocity, battery level)
 - environment properties (object locations, obstacles)
 - hidden properties (intentions, goals, failures)
- Different models may use different state representations.

Partial observability means that some components of the chosen state representation are unknown.

Latent variables are hidden variables that affect the evolution of the system but are not directly observable.

Partial Observability and Partial Knowledge

Uncertainty can arise from several sources:

- **Unknown state variables**
 - Where is the target?
 - Which doors are locked?
- **Incomplete environment models**
 - Unknown transition probabilities
 - Unknown rewards or costs
- **Unknown agent models**
 - Human intentions
 - Behavior of other agent
- ...

Many AI problems can be viewed as decision-making under incomplete knowledge.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Example

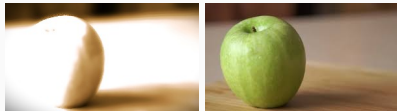
Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Partial and noisy sensing



Partial observability over time



Example

Autonomous Driving

- The vehicle never observes the full traffic state.
- Other drivers' intentions are hidden.
- Future events are uncertain.



Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

What if we ignore relevant information and plan
with what we know?

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Partial Observability as Noise

What if we ignore relevant information and plan with what we know?

For example, what would happen if we try to compute a policy only based on the current image? Assume the task is to collect green apples, but there is no ability to know the color of the object.



What is a relevant policy?

What does that mean in terms of $V_{\pi}(s)$?

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Partial Observability as Noise

The same observation may correspond to multiple hidden states: some hidden states yield high return (green apple), while others yield low return (red apple). The agent (and policy) cannot distinguish between them.



From the agent's perspective, hidden variables yield **noise** and **variability**.

$$V^{\pi}(o) = \mathbb{E}[V^{\pi}(s) \mid o]$$

$$V^*(o) = \max_{a \in \mathcal{A}} \mathbb{E}[Q^*(s, a) \mid o]$$

Hidden variables turns state uncertainty into outcome variability: the value associated with an observation becomes an average over many possible underlying states.

Partial Observability and Partial Knowledge

Not all uncertainty comes from missing sensor information.

Many important factors affecting decisions cannot be directly observed or accurately modeled.

- Human intentions and preferences
- Future actions of other agents
- Political and regulatory changes
- Economic and societal trends
- Rare events and system failures
- Unknown consequences of actions

Examples ?

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Partial Observability and Partial Knowledge

- A self-driving car cannot directly observe the intentions of nearby drivers.
- An energy operator cannot perfectly predict future demand or policy changes.
- A company cannot fully anticipate market reactions to a new product.



Partial knowledge extends beyond sensing: the agent may lack information about the environment, other agents, or the processes governing future outcomes.

Partial Observability and Reinforcement Learning

In reinforcement learning (our next topic), we often do not know everything required to make optimal decisions.

This uncertainty can arise from:

- Hidden state variables
- Unknown transition dynamics
- Unknown reward functions
- Behavior of other agents
- Unobserved intentions of other agents

In principle, many of these uncertainties can be modeled as forms of partial observability.

Even if the physical state is fully observable, uncertainty about the reward function, transition function, or other agents can be viewed as hidden information and incorporated into an expanded state representation.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

POMDP

Partial Observability via POMDPs

The way we represent partial observability is a modeling choice rather than a property of the world itself.

We can model hidden information explicitly using a POMDP. Alternatively, we can absorb that uncertainty into stochastic transitions and use an MDP.

In a POMDP, we explicitly distinguish between:

- the **true state** of the world
- the agent's **observations** of that state via a stochastic sensor function $o_t \sim \mathcal{O}(s_t)$.

The agent must reason about which states are consistent with its observations.

There is often no sharp distinction between uncertainty caused by hidden states and uncertainty caused by stochastic dynamics. A POMDP makes the hidden variables explicit.

Regret in Partially Observable Domains

Unlike fully observable settings, regret arises from two sources:

1 Learning uncertainty

The agent does not yet know the environment model.

2 State uncertainty

Even with a perfect model, the true state is hidden and must be inferred from observations.

$$\text{Regret} = \underbrace{\text{Learning error}}_{\text{unknown model}} + \underbrace{\text{Information limitations}}_{\text{partial observability}}$$

Should the agent gather reward now, or collect information that may improve future decisions?

This trade-off is the **exploration-exploitation-information gathering trade-off**.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Regret in POMDPs

In a POMDP, decisions are made using the observation history

$$h_t = (o_0, a_0, o_1, a_1, \dots, o_t)$$

or equivalently the belief state

$$\beta_t(s) = \mathcal{P}(s_t = s \mid h_t).$$

The regret of a policy π is

$$R_T = V_T^{\pi^*}(\beta_0) - V_T^\pi(\beta_0),$$

where π^* is the optimal policy over belief states.

The benchmark is no longer the optimal policy for the true state, but the optimal policy given the available information.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

MCTS in POMDPs

Key Idea: use **Monte Carlo simulation** to accumulate value estimates to guide towards highly rewarding trajectories in a search tree.

- Each simulation consists of two phases
 - **Tree policy:** pick actions to maximise values
 - **Default / roll-out policy:** pick actions randomly to simulate a trajectory.
- Repeat (each simulation)
 - **Evaluate** states $Q(s, a)$ by Monte-Carlo evaluation
 - **Improve** tree policy e.g. by ϵ -greedy.

Monte Carlo Tree Search- Reminder

function MONTE-CARLO-TREE-SEARCH(*state*) **returns** an action

$tree \leftarrow \text{NODE}(state)$

while IS-TIME-REMAINING() **do**

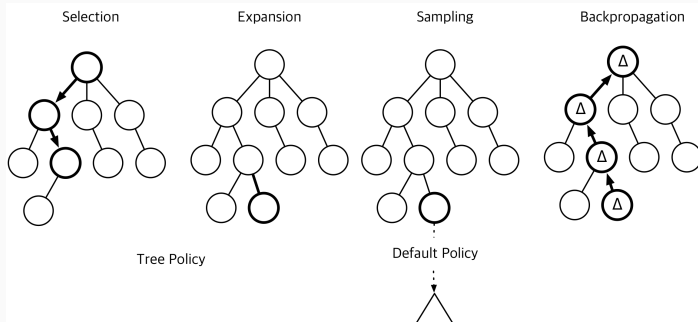
$leaf \leftarrow \text{SELECT}(tree)$

$child \leftarrow \text{EXPAND}(leaf)$

$result \leftarrow \text{SIMULATE}(child)$

$\text{BACK-PROPAGATE}(result, child)$

return the move in $\text{ACTIONS}(state)$ whose node has highest number of playouts



Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Reminder: Partially Observable Markov Decision Process (POMDP)

Observability

POMDP

MCTS in POMDPs

Conclusion

A **Partially Observable Markov Decision Process**(POMDP) is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \Omega, \mathcal{O}, \beta_0 \rangle$ where

- $\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}$ and γ are as for an MDP.
- Ω is a set of observations (observation tokens),
- $\mathcal{O}$ is a sensor function specifying the conditional observation probabilities $\mathcal{O}_{s,a}^o = \mathcal{P}[O_{t+1} = o | S_t = s, A_t = a]$ of receiving observation token $o \in \Omega$ in state s after applying action a ¹.
- β_0 the initial belief: a probability distribution over the states such that $\beta_0(s)$ stands for the probability of s being the true initial state.

¹alternatively: $\mathcal{O}_s^o = \mathcal{P}[O_t = o | S_t = s]$

Recall that a belief is a probability distribution over states:

$$\beta(s) = \mathcal{P}(s \mid h)$$

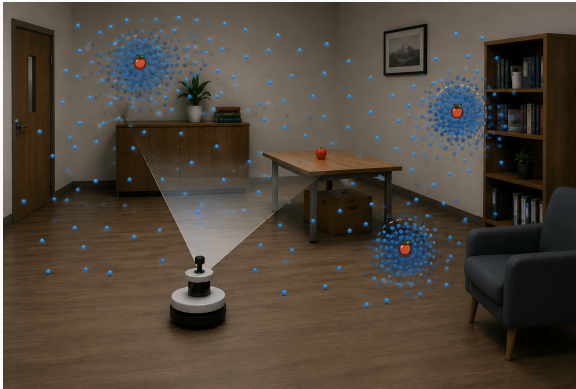
where h is the observation history.

For large state spaces, representing the full belief explicitly is often infeasible. This can be addressed by approximating the belief using a set of sampled states (particles):

$$\beta \approx \{s^{(1)}, s^{(2)}, \dots, s^{(N)}\}$$

Belief Representation with Particles

- Each particle is a possible hidden state.
- More particles in a region indicate higher probability.
- Simulations start from sampled particles rather than the full belief.



Particles provide a compact approximation of the belief state,

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Particles in POMDP Solution

Classical POMDP solvers maintain and update beliefs explicitly.

$$\beta_t \rightarrow \beta_{t+1}$$

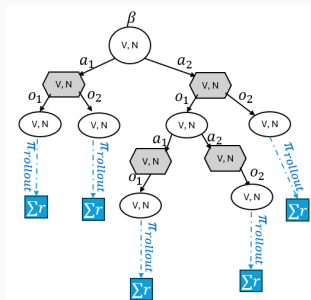
POMCP avoids explicit belief updates.

Instead:

- 1 Sample a particle $s \sim \beta(h)$
- 2 Simulate trajectories from that state.
- 3 Store the particle at the corresponding history node.
- 4 Reuse accumulated particles in future searches.

The search tree itself becomes a particle-based belief representation. Each history node stores states that are consistent with that history.

- Extending MCTS to POMDPs.
- Relevant to large or continuous POMDPs where explicit belief updates are intractable.
- Basic idea: use Monte-Carlo in two ways:
 - sampling start states from the belief.
 - sampling histories using a black box simulator.
- **Online algorithm:** iteratively expands a search tree of histories.



Key assumption:

- Access to a *black-box generative model*

$$(s', o, r) \sim G(s, a)$$

- No explicit representation of $\mathcal{P}$ or $\mathcal{O}$

Goal: Select next action from the current belief without enumerating states or observations.

POMCP builds a search tree over *histories*:

$$h = (a_1, o_1, \dots, a_t, o_t)$$

Belief *implicitly* represented using particles:

$$\beta \approx \{s^{(1)}, s^{(2)}, \dots, s^{(N)}\}$$

Tree structure:

- Nodes correspond to beliefs induced by histories
- Actions branch to observation nodes via sampling

Simulation loop:

- 1 Sample a particle $s \sim \beta(h)$
- 2 Select action using selection rule.
 - e.g., UCB : $a \leftarrow \arg \max_a \left(Q, a \right) + c \sqrt{\frac{\ln N(h)}{N(h, a)}}$
- 3 Sample $(s', o, r) \sim G(s, a)$ and rollout

Statistics are propagated upwards to estimate action values Q, a).

POMCP (Silver & Veness)

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

function POMCP(β, n)

Require: Belief β , number of simulations n

for $i = 1$ to n **do**

$s \leftarrow \text{SAMPLE}(\beta)$

 SIMULATE($s, \emptyset, 0$)

end for

return $\arg \max_a Q(h_{\text{root}}, a)$

end function

POMCP (Silver & Veness)

```
function SIMULATE( $s, h, d$ )  
  if  $d = D$  then  
    return 0  
  end if  
  if  $h \notin \mathcal{T}$  then  
    Add node  $h$  to tree  $\mathcal{T}$   
    return ROLLOUT( $s, h, d$ )  
  end if  
   $a \leftarrow \text{UCB}(h)$   
   $(s', o, r) \leftarrow \text{SIMULATEENV}(s, a)$   
   $h' \leftarrow h \frown (a, o)$   
   $q \leftarrow r + \gamma \cdot \text{SIMULATE}(s', h', d + 1)$   
   $N(h) \leftarrow N(h) + 1$   
   $N(h, a) \leftarrow N(h, a) + 1$   
   $Q, a \leftarrow Q, a + \frac{q - Q, a}{N(h, a)}$   
  return  $q$   
end function
```

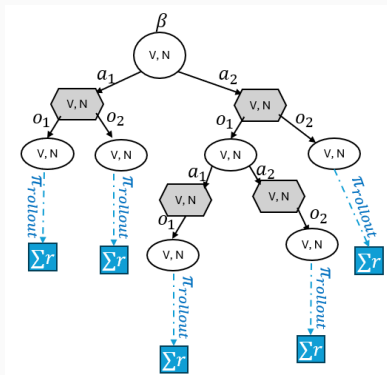
Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

```
function ROLLOUT( $s, h, d$ )  
  if  $d = D$  then  
    return 0  
  end if  
   $a \leftarrow \text{ROLLOUTPOLICY}$   
   $(s', o, r) \leftarrow \text{SIMULATEENV}(s, a)$   
  return  $r + \gamma \cdot \text{ROLLOUT}(s', h \frown (a, o), d + 1)$   
end function
```



Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Strengths:

- Scales to large state spaces
- Avoids explicit belief representations
- Well-suited for online planning

Limitations:

- Performance depends on quality of generative model
- Observation abstraction errors propagate to planning
- Requires many simulations for deep uncertainty

Careful integration of perception and belief sampling is essential.

Why Does POMCP Scale?

Classical POMDP algorithms reason explicitly over belief states.

$$\beta(s) = \mathcal{P}(s \mid h)$$

This becomes difficult because:

- Belief spaces are continuous
- The number of reachable histories grows exponentially
- Exact belief updates can be expensive

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Why Does POMCP Scale?

Classical POMDP algorithms reason explicitly over belief states.

$$\beta(s) = \mathcal{P}(s \mid h)$$

This becomes difficult because:

- Belief spaces are continuous
- The number of reachable histories grows exponentially
- Exact belief updates can be expensive

POMCP avoids these costs by:

- Representing beliefs with particles
- Sampling trajectories instead of enumerating them
- Building only the relevant part of the search tree

POMCP replaces exact belief-space reasoning with Monte Carlo approximation.

What Is Being Approximated?

POMCP performs approximation at several levels.

Belief approximation

$$\beta \approx \{s^{(1)}, \dots, s^{(N)}\}$$

Tree approximation

- Explore only histories visited by simulations

Value approximation

- Estimate action values using sampled returns

$$Q(h, a) \approx \frac{1}{N(h, a)} \sum_i R_i$$

POMCP trades computation for statistical estimation.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

The Curse of Dimensionality vs. The Curse of History

Classical POMDP planning suffers from two major challenges.

Curse of Dimensionality

- Belief states are continuous distributions
- Complexity grows with the number of states

Curse of History

- Number of possible histories grows exponentially
- Observation branching creates enormous trees

POMCP mitigates both curses through sampling: it samples states, observations, and trajectories instead of enumerating them.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

POMCP as a Belief-Space Planner

Recall:

$$\pi^*(b) = \arg \max_a Q^*(b, a)$$

POMCP never explicitly computes:

- belief updates,
- belief transitions,
- belief value functions.

Instead it performs Monte Carlo search directly on histories.

$$h = (a_1, o_1, \dots, a_t, o_t)$$

POMCP is still solving a belief-space planning problem, but it does so implicitly through sampling.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Discussion: When Would You Use POMCP?

Good fit

- Large state spaces
- Complex observation models
- Online decision making
- Access to a simulator

Poor fit

- No generative model available
- Very long horizons
- Severe real-time constraints
- Continuous observations without progressive widening

POMCP is often the method of choice when exact POMDP planning is impossible but a simulator is available.

POMCP Limitations

POMCP assumes that observations can be represented by a finite set of observation tokens.

Modern AI systems often violate this assumption:

- Images
- Video streams
- Natural language
- High-dimensional sensor data

Furthermore:

- The simulator may be inaccurate
- Rare events may not be sampled
- Planning becomes expensive for long horizons

Modern extensions combine POMCP with learned representations, value functions, and structured belief models.

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

POMCP: Algorithmic Extensions*

Goal: Improve scalability, convergence, and robustness of online belief-space planning.

- **Progressive Widening (DPW)**
 - Controls branching in large or continuous action / observation spaces
- **Factored & Structured Beliefs**
 - Exploit conditional independence in the state
 - Particle beliefs over factored variables
- **Guided Rollouts & Value Priors**
 - Replace random rollouts with heuristics or learned policies
 - Can incorporate value networks or domain knowledge
- **Learning-Augmented POMCP**
 - Online learning of transition, observation, or reward models
 - Bridges model-based planning and reinforcement learning
- **Risk-Sensitive Objectives**
 - Optimize CVaR, chance constraints, or worst-case value
 - Supports safety-aware decision making

Learning under
Partial
Observability

POMDP

MCTS in POMDPs

Conclusion

Conclusion

- **Classical considerations:** completeness, optimality, time complexity, space complexity, anytime guarantees
- **Modern considerations:** robustness, scalability, adaptivity, generality, interpretability, energy efficiency

Where do the methods we have seen stand?

- POMDPs model decision making when the true state is not fully observable.
- Exact belief-space planning is often computationally intractable.
- We explored POMCP that combines particle-based belief representation with MCTS: online planning from the current belief.
- This demonstrates planning through simulation rather than explicit belief-state enumeration.
- The approach scales to large state spaces and complex observation models when a simulator is available.

POMCP demonstrates a powerful idea: **approximate reasoning through sampling can make planning under uncertainty practical.**